

Informatikarbeit Nr. 1 Dr. Winkelmann	Klasse 10G	Name	Datum
<i>Suchen und Sortieren</i>			
Punktzahl: von 55 Punkten			Note:

Arbeitszeit: 90 Minuten · Hilfsmittel: keine

1. **Grundbegriffe** In dieser Arbeit geht es um das Suchen und das Sortieren von Daten.

a. **Ordne** jedem Verfahren die passende Aussage zu. Schreibe die Paare auf, z. B. A – 3. (4 BE)

b. **Verfahren:**

A	Lineare	Suche
B	Binäre	Suche
C	Bubble	Sort

Aussagen:

- 1 vertauscht benachbarte Elemente, bis die Liste sortiert ist
 2 prüft die Einträge der Reihe nach vom ersten bis zum letzten
 3 halbiert bei jedem Schritt den noch zu durchsuchenden Bereich

c. **Gib** an, welche Voraussetzung eine Liste erfüllen muss, damit die binäre Suche überhaupt verwendet werden darf. (3 BE)

2. **Lineare Suche** Die folgende *unsortierte* Liste wird mit der linearen Suche durchsucht. Gesucht wird die Zahl **8**.

Liste:	7	3	9	2	8	5
Index:	1	2	3	4	5	6

a. **Analysiere** den Ablauf der linearen Suche. Schreibe der Reihe nach auf, mit welchen Werten die gesuchte Zahl verglichen wird, bis sie gefunden ist. (4 BE)

b. **Gib** an, wie viele Vergleiche die lineare Suche bei dieser Liste im ungünstigsten Fall benötigt. (2 BE)

3. **Binäre Suche** Die folgende *sortierte* Liste wird mit der binären Suche durchsucht. Gesucht wird die Zahl **17**. Bei gerader Elementzahl wird als Mitte das linke der beiden mittleren Elemente gewählt.

Liste:	2	5	8	11	17	23	30
Index:	1	2	3	4	5	6	7

a. **Analysiere** die binäre Suche schrittweise. Gib für jeden Schritt an, welches Element als Mitte verglichen wird und in welcher Hälfte die Suche weitergeht, bis die 17 gefunden ist. (5 BE)

b. **Berechne** mithilfe von $\log_2(n)$, wie viele Vergleiche die binäre Suche bei einer sortierten Liste mit $n = 1000$ Einträgen höchstens benötigt. (3 BE)

4. **Bubble Sort durchführen** Bubble Sort sortiert die folgende Liste aufsteigend. Es werden immer zwei benachbarte Elemente verglichen und getauscht, wenn das linke größer ist.

Start:	5	1	4	2	3
Index:	1	2	3	4	5

a. **Stelle** den Zustand der Liste nach dem ersten und nach dem zweiten vollständigen Durchlauf dar. Trage die Werte in die leeren Kästchen ein. (6 BE)

Durchlauf:					
------------	--	--	--	--	--

Durchlauf:					
------------	--	--	--	--	--

b. **Ermittle** die Anzahl der Vergleiche, die im ersten Durchlauf durchgeführt werden. (2 BE)

5. **Bubble Sort vervollständigen** Der folgende Pseudocode soll eine Liste a mit n Elementen aufsteigend sortieren. An drei Stellen fehlt etwas (___).
- ```
wiederhole solange noch getauscht wurde:
 für j von 1 bis ___ : (1)
 wenn $a[j]$ ___ $a[j+1]$: (2)
 ___ (3)
```
- a. **Implementiere** die fehlenden Teile. Gib für die Stellen (1), (2) und (3) jeweils an, was dort stehen muss, damit der Algorithmus korrekt aufsteigend sortiert. Pseudocode genügt. (6 BE)
6. **Fehler in der linearen Suche** Der folgende Pseudocode soll prüfen, ob die gesuchte Zahl  $x$  in der Liste  $a$  vorkommt. Er enthält einen Logikfehler: Bei vielen Eingaben meldet er fälschlich „nicht gefunden“, obwohl die Zahl enthalten ist.
- ```
1 funktion enthält( $a, n, x$ ):
2 für  $i$  von 1 bis  $n$ :
3 wenn  $a[i] = x$ :
4 gib WAHR zurück
5 sonst:
6 gib FALSCH zurück
```
- a. **Korrigiere** den Pseudocode. Gib an, in welcher Zeile der Fehler liegt, und schreibe die berichtigte Fassung des Algorithmus auf. (5 BE)
7. **Warum steigt das Größte auf?**
- a. **Begründe**, warum bei Bubble Sort nach dem ersten vollständigen Durchlauf das größte Element der Liste sicher ganz rechts steht. (4 BE)
8. **Zusatz: Welches Verfahren für die Mediathek?** Eine Video-Mediathek verwaltet rund **1 000 000** alphabetisch sortierte Filmtitel. Ein Nutzer sucht einen bestimmten Titel.
- a. **Berechne** für beide Verfahren die Anzahl der Vergleiche im ungünstigsten Fall: für die lineare Suche und – mithilfe von $\log_2(n)$ – für die binäre Suche (auf eine ganze Zahl aufgerundet). (5 BE)
- b. **Beurteile**, welches der beiden Suchverfahren für die Mediathek geeignet ist. Begründe deine Entscheidung mit den berechneten Zahlen und gehe darauf ein, warum sich das vorherige Sortieren der Titel lohnt. (6 BE)

Verwendete Operatoren

Operator	Beschreibung der erwarteten Leistung
zuordnen	Elemente, Begriffe oder Darstellungen nach vorgegebenen Kriterien in Beziehung setzen
angeben	Sachverhalte, Zusammenhänge, Methoden ohne nähere Erläuterungen, Begründungen und ohne Darstellung des Lösungswegs wiedergeben
analysieren	gegebenen Quelltext oder ein gegebenes Modell schrittweise nachvollziehen und z. B. Ausgabe, Variablenbelegung oder Verhalten ermitteln
berechnen	numerische Ergebnisse von einem Ansatz ausgehend gewinnen
darstellen	Sachverhalte, Zusammenhänge oder Daten strukturiert in eine geeignete (andere) Form übertragen, z. B. Wertetabelle in einen Graphen
ermitteln	einen Zusammenhang oder ein Ergebnis mit einem geeigneten Verfahren gewinnen; der Lösungsweg muss erkennbar sein
implementieren	einen Algorithmus oder eine Datenstruktur in einer Programmiersprache (oder in Pseudocode) lauffähig umsetzen
korrigieren	Fehler in gegebenem Quelltext oder Modell lokalisieren und beheben
begründen	Sachverhalte auf Regeln und Gesetzmäßigkeiten bzw. kausale Zusammenhänge zurückführen
beurteilen	zu einem Sachverhalt ein selbstständiges Urteil unter Verwendung von Fachwissen und Fachmethoden formulieren und begründen

Erwartungshorizont zur Informatikarbeit Nr. 1

Aufgab e	Erwartete Leistung	AFB	Soll	Ist										
1a	<i>Zuordnen:</i> richtige Paare: ** A – 2 (lineare Suche: der Reihe nach), B – 3 (binäre Suche: halbiert) ** C – 1 (Bubble Sort: benachbarte Elemente tauschen) (je richtigem Paar 1 BE; 4 BE bei allen drei korrekt und sauber notiert)	I	4											
1b	<i>Angeben:</i> *** Die Liste muss sortiert sein (z. B. aufsteigend geordnet). (Kernaussage „sortiert“ 2 BE; passende Ergänzung/Beispiel 1 BE)	I	3											
2a	<i>Analysieren:</i> Vergleich der Reihe nach mit **** 7 (≠), 3 (≠), 9 (≠), 2 (≠), 8 (= gefunden) (4 korrekte Vergleichswerte in richtiger Reihenfolge bis zum Treffer)	I	4											
2b	<i>Angeben:</i> ** 6 Vergleiche (alle $n = 6$ Elemente im ungünstigsten Fall).	I	2											
3a	<i>Analysieren:</i> sortierte Liste [2, 5, 8, 11, 17, 23, 30]: * Schritt 1: Mitte = 11 (Index 4); $17 > 11 \rightarrow$ rechte Hälfte [17, 23, 30] * Schritt 2: Mitte = 23 (Index 6); $17 < 23 \rightarrow$ linke Hälfte [17] * Schritt 3: Mitte = 17 (Index 5); $17 = 17 \rightarrow$ gefunden (je korrektem Mittelwert 1 BE; richtige Hälften-Entscheidungen 2 BE)	II	5											
3b	<i>Berechnen:</i> * $\log_2(1000) \approx 9,97$; * Aufrunden auf 10. * Ergebnis: höchstens 10 Vergleiche. (auch zulässig: Halbieren $1000 \rightarrow 500 \rightarrow \dots \rightarrow 1$ in 10 Schritten)	II	3											
4a	<i>Darstellen:</i> Start [5, 1, 4, 2, 3] *** nach 1. Durchlauf: [1, 4, 2, 3, 5] *** nach 2. Durchlauf: [1, 2, 3, 4, 5]	II	6											
	<i>Lösungsbild:</i> Durchlauf: <table border="1" style="display: inline-table; vertical-align: middle;"><tr><td>1</td><td>4</td><td>2</td><td>3</td><td>5</td></tr></table> Durchlauf: <table border="1" style="display: inline-table; vertical-align: middle;"><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	1	4	2	3	5	1	2	3	4	5		0	
1	4	2	3	5										
1	2	3	4	5										
4b	<i>Ermitteln:</i> ** 4 Vergleiche ($n - 1 = 5 - 1$).	I	2											
5	<i>Implementieren:</i> fehlende Teile: ** (1) $n - 1$ bzw. $n-1$ (innere Schleife läuft bis zum vorletzten Element) ** (2) $a[j] > a[j+1]$ ** (3) vertausche $a[j]$ und $a[j+1]$ (je Lücke 2 BE; Logik zählt, Syntaxdetails sind unerheblich. Eine kleinere obere Grenze mit korrekter Begründung wird akzeptiert.)	II	6											
6	<i>Korrigieren:</i> * Fehler in Zeile 5/6: das sonst: gib FALSCH zurück steht innerhalb der Schleife. * Dadurch bricht die Suche schon nach dem ersten Element ab; FALSCH darf erst nach der Schleife stehen. *** berichtigte Fassung (Logik): für i von 1 bis n : wenn $a[i] = x$: gib WAHR zurück gib FALSCH zurück (erst NACH der Schleife) (Fehlerstelle 1 BE, Begründung 1 BE, korrekte Struktur 3 BE)	II	5											
7	<i>Begründen:</i> * In jedem Schritt wird das größere zweier benachbarter Elemente nach rechts getauscht. * Das aktuell größte Element gewinnt jeden Vergleich und wird daher fortlaufend weitergeschoben. ** Nach einem vollständigen Durchlauf hat es so die letzte (rechte) Position erreicht.	III	4											
8a	<i>Berechnen:</i> ** lineare Suche: bis zu 1 000 000 Vergleiche (alle n Einträge). $\log_2(1000000) \approx 19,93$; *** binäre Suche: höchstens 20 Vergleiche.	II	5											

8b	<i>Beurteilen:</i> ** Die binäre Suche ist geeignet: 20 statt bis zu 1 000 000 Vergleiche – enormer Unterschied. ** Die Liste ist sortiert, daher ist die binäre Suche überhaupt anwendbar. ** Das einmalige Sortieren lohnt sich, weil danach jede spätere Suche extrem schnell ist. (Entscheidung mit Zahlbezug 3 BE; Voraussetzung sortiert 1 BE; Nutzen des Sortierens 2 BE)	III	6	
		ges.	55	

Folgefehler: Ein bereits sanktionierter Fehler führt in Folgeschritten nicht zu erneutem Abzug, sofern fachgerecht weitergerechnet wurde.

Fachlich korrekte alternative Lösungswege werden gleichwertig bepunktet.

Bewertet wird die Logik des Algorithmus; Syntaxdetails (z. B. fehlende Doppelpunkte, Klammern) führen nicht zu Abzug, solange die Absicht eindeutig ist.

Benotung

Note	1	2	3	4	5	6
ab BE	49,5	43	34,5	27,5	11	< 11